

Clustering

العنقدة

1. What is cluster analysis?

Cluster analysis or (clustering) is the process of partitioning a set of data objects (or observations) into subsets. Each subset is a **cluster**, such that objects in a cluster are similar to one another, but are dissimilar to objects in other clusters. The set of clusters resulting from a cluster analysis can be referred to as a **clustering**.

The similarity between data objects is usually measured using a distance or similarity metric based on features, and no prior class labels are required.

Clustering as a data mining tool has its roots in many application areas, such as biology, security, business intelligence, and Web search.

Clustering is sometimes called **automatic classification** or **unsupervised classification**.

Clustering is known as **unsupervised learning** because the class label information is not present.

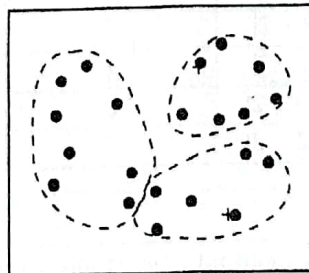
2. Basic clustering methods

Clustering techniques can be broadly organized into several fundamental types.

- Partitioning methods** divide a dataset of n objects into k clusters, where each object belongs to exactly one cluster and k is typically much smaller than n . Such as **k-means** and **k-medoids** are commonly used. These methods work well for spherical clusters in small to medium-sized datasets but require extensions for complex shapes, large datasets, or high-dimensional data.
- Hierarchical methods** create a hierarchy of clusters, represented as a tree structure. They can be agglomerative (bottom-up), starting with each object as a separate cluster and merging them, or divisive (top-down), starting with one cluster and repeatedly splitting it. Their main limitation is that once clusters are merged or split, the decision cannot be undone, which reduces flexibility but lowers computational cost.
- Density-based and grid-based methods** were developed to overcome the limitations of Partitioning-based approaches. Density-based clustering forms clusters by identifying dense regions of data points and can discover clusters of arbitrary shapes while filtering out noise and outliers. Grid-based methods implement density concepts by partitioning the data space into a finite grid of cells and performing clustering on these cells rather than individual objects. Their key advantage is efficiency, as processing time depends on the number of grid cells rather than the number of data points. These methods can also be combined with other clustering techniques, such as hierarchical clustering.

3. k-Means: a centroid-based technique

k-Means is a **centroid-based partitioning clustering technique** that divides a dataset of n objects in Euclidean space into k non-overlapping clusters, where each cluster contains at least one object. The goal is to achieve **high similarity within clusters** and **low similarity between clusters**, which is evaluated using an objective function.



In centroid-based clustering, each cluster is represented by a **centroid**, which conceptually corresponds to the center of the cluster. The centroid is typically defined as the **mean** (or sometimes the medoid) of the data points assigned to the cluster. The distance between a data point and its cluster centroid is measured using **Euclidean distance**.

How does the k-means algorithm work?

- k-means is a centroid-based clustering algorithm.
- Each cluster is represented by a centroid, defined as the mean of its data points.
- The algorithm starts by randomly selecting k data points as initial centroids.
- Each data point is assigned to the nearest centroid using Euclidean distance.
- New centroids are computed by calculating the mean of all points in each cluster.
- Data points are reassigned based on the updated centroids.
- The assignment and update steps are repeated iteratively.
- The algorithm stops when cluster assignments no longer change (convergence).

Algorithm: k-means. The k -means algorithm for partitioning, where each cluster's center is represented by the mean value of the objects in the cluster.

Input:

- k : the number of clusters,
- D : a data set containing n objects.

Output: A set of k clusters.

Method:

- (1) arbitrarily choose k objects from D as the initial cluster centers;
- (2) **repeat**
- (3) (re)assign each object to the cluster to which the object is the most similar;
- (4) update the cluster centers, that is, calculate the mean value of the objects for each cluster;
- (5) **until** no change;

Example: k-Means in One Dimension (1D)

Given Data points: $\{2, 4, 6, 10, 12\}$ Number of clusters: $k = 2$

Distance Formula: $d(x, c) = |x - c|$

Step 1: Choose Initial Centroids (Randomly)

Let:

- Centroid 1 = 2
- Centroid 2 = 10

Step 2: First Assignment

Data Point (x)	Distance to C1 = 2	Calculation	Distance to C2 = 10	Calculation	Assigned Cluster
2	0	$ 2 - 2 $	8	$ 2 - 10 $	C1
4	2	$ 4 - 2 $	6	$ 4 - 10 $	C1
6	4	$ 6 - 2 $	4	$ 6 - 10 $	C1
10	8	$ 10 - 2 $	0	$ 10 - 10 $	C2
12	10	$ 12 - 2 $	2	$ 12 - 10 $	C2

Step 3: Compute New Centroids

Cluster	Points	Centroid Calculation	New Centroid
C1	2, 4, 6	$(2 + 4 + 6) \div 3$	4
C2	10, 12	$(10 + 12) \div 2$	11

Step 4: Reassignment

Data Point (x)	Distance to C1 = 4	Calculation	Distance to C2 = 11	Calculation	Assigned Cluster
2	2	$ 2 - 4 $	9	$ 2 - 11 $	C1
4	0	$ 4 - 4 $	7	$ 4 - 11 $	C1
6	2	$ 6 - 4 $	5	$ 6 - 11 $	C1
10	6	$ 10 - 4 $	1	$ 10 - 11 $	C2
12	8	$ 12 - 4 $	1	$ 12 - 11 $	C2

Final Result

- Cluster 1: {2, 4, 6}
- Cluster 2: {10, 12}

Exam Question (k-Means Clustering)

Given the following two-dimensional dataset:

Point	x	y
A	1	1
B	2	1
C	2	2
D	6	4
E	7	5
F	8	5
G	9	6
H	5	3

Let the number of clusters be $k = 2$.

1. Use the k-means algorithm with the Manhattan (absolute) distance

$$d((x, y), (c_x, c_y)) = |x - c_x| + |y - c_y|$$

2. Assume the initial centroids are:

- $(C_1 = (6, 4))$
- $(C_2 = (9, 6))$

3. Perform at least three iterations.

4. Show the final clusters.

Step 1: Initialization

We start by randomly selecting initial centroids:

- Cluster 1 centroid: $(C_1 = (6, 4))$
- Cluster 2 centroid: $(C_2 = (9, 6))$

These centroids represent the initial "centers" of the clusters.

Iteration 1**Step 2: Assignment Phase**

Each data point is assigned to the cluster whose centroid is **closest**, based on Manhattan distance.

Example calculation:

For point A(1,1)

- Distance to (C_1) : $(|1-6| + |1-4| = 5 + 3 = 8)$
 - Distance to (C_2) : $(|1-9| + |1-6| = 8 + 5 = 13)$
- Assign A to Cluster 1

Applying the same process to all points:

Point	d to C1	d to C2	Cluster
A	8	13	C1
B	7	12	C1
C	6	11	C1
D	0	5	C1
E	2	3	C1
F	3	2	C2
G	5	0	C2
H	2	7	C1

Step 3: Update Phase (Recalculate Centroids)

The new centroid of each cluster is calculated as the **mean of all points in that cluster**.

Cluster 1: A, B, C, D, E, H

$$C_1 = \left(\frac{1 + 2 + 2 + 6 + 7 + 5}{6}, \frac{1 + 1 + 2 + 4 + 5 + 3}{6} \right) = (3.83, 2.67)$$

Cluster 2: F, G

$$C_2 = \left(\frac{8 + 9}{2}, \frac{5 + 6}{2} \right) = (8.5, 5.5)$$

Iteration 2

Assignment Phase

Distances are recomputed using the updated centroids.

Key change:

- Point E(7,5) is now closer to (C_2) than (C_1)

Point	d to C1	d to C2	Cluster
A	4.50	12.00	C1
B	3.50	11.00	C1
C	2.50	10.00	C1
D	3.50	4.00	C1
E	5.50	2.00	C2
F	6.50	1.00	C2
G	8.50	1.00	C2
H	1.50	6.00	C1

Update Phase

Cluster 1: A, B, C, D, H

$$C_1 = (3.2, 2.2)$$

Cluster 2: E, F, G

$$C_2 = (8, 5.33)$$

Iteration 3

Assignment Phase

Another change occurs:

- Point D(6,4) moves from Cluster 1 to Cluster 2

Point	d to C1	d to C2	Cluster
A	3.40	11.33	C1
B	2.40	10.33	C1
C	1.40	9.33	C1
D	4.60	3.33	C2
E	6.60	1.33	C2
F	7.60	0.33	C2
G	9.60	1.67	C2
H	2.60	5.33	C1

Update Phase

- Cluster 1 $\rightarrow$ A, B, C, H
 $C_1 = (2.5, 1.75)$
- Cluster 2 $\rightarrow$ D, E, F, G
 $C_2 = (7.5, 5)$

Iteration 4 (Convergence Check)

All points remain in the same clusters.

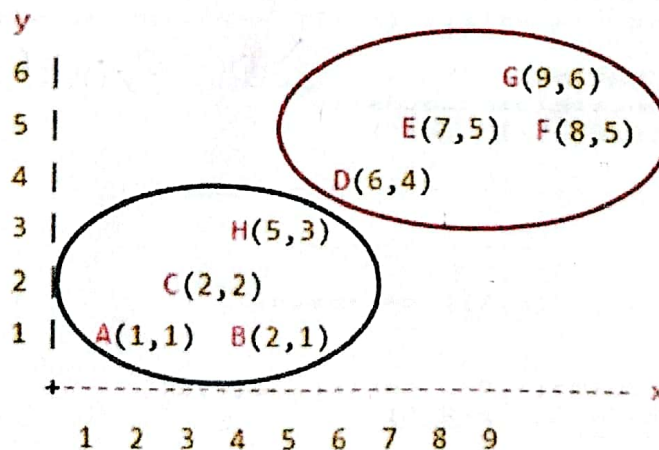
No changes $\rightarrow$ algorithm converges

حسابات / تبليغات

Final Clusters

- Cluster 1: A(1,1), B(2,1), C(2,2), H(5,3)
- Cluster 2: D(6,4), E(7,5), F(8,5), G(9,6)

Not changed so we stopped here




```

import numpy as np
import matplotlib.pyplot as plt
from sklearn.cluster import KMeans
# -----
# Data points (same example)
# -----
X = np.array([
    [1, 1], # A
    [2, 1], # B
    [2, 2], # C
    [6, 4], # D
    [7, 5], # E
    [8, 5], # F
    [9, 6], # G
    [5, 3] # H
])
names = ['A', 'B', 'C', 'D', 'E', 'F', 'G', 'H']
# Initial centroids
C = np.array([[6, 4], [9, 6]])
# -----
# Apply k-Means
# -----
kmeans = KMeans(n_clusters=2, init=C, n_init=1, random_state=0)
kmeans.fit(X)
labels = kmeans.labels_
centroids = kmeans.cluster_centers_
# -----
# Printing Results
# -----
print("Data points and their clusters:\n")
for i in range(len(X)):
    print(f"Point {names[i]} {X[i]} --> Cluster {labels[i]}")
print("\nFinal Centroids:")
for i, c in enumerate(centroids):
    print(f"Centroid {i}: {c}")
# -----
# Plotting
# -----
plt.figure()
plt.scatter(X[:,0], X[:,1], c=labels)

# Plot centroids
plt.scatter(centroids[:,0], centroids[:,1],
            marker='X', s=200)

# Label points
for i, txt in enumerate(names):
    plt.text(X[i,0]+0.1, X[i,1]+0.1, txt)

plt.xlabel("X")
plt.ylabel("Y")
plt.title("k-Means Clustering Example (2D)")
plt.grid(True)
plt.show()

```

Print(labels[i]) ^{مؤشر}

Print(f"cluster {labels[i]}")

الكود
المتروك

kmeans = KMeans(n_clusters=2, init=C, n_init=1, random_state=0)

ماذا يعني هذا السطر؟

• KMeans

← استدعاء خوارزمية k-means من مكتبة sklearn.

• n_clusters=2

← عدد العناقيد = 2 (Clusters)

أي نريد تقسيم البيانات إلى مجموعتين.

• init=C

← تحديد المراكز الابتدائية (Initial Centroids) يدويًا.

في مثالنا:

• $C = [[6,4], [9,6]]$

وهذا مهم للتوضيح التعليمي لأن نتائج k-means تعتمد على البداية.

• n_init=1

← عدد مرات تشغيل الخوارزمية ببدايات مختلفة.

وضعناها = 1 لأننا حددنا البداية بأنفسنا.

• random_state=0

← لتثبيت العشوائية وضمان الحصول على نفس النتيجة في كل تشغيل.

kmeans.fit(X)

ماذا يفعل هذا السطر؟

• يطبق خوارزمية k-means على البيانات X.

• الخوارزمية تقوم تلقائيًا بـ:

1. حساب المسافة بين كل نقطة وكل مركز

2. إسناد النقاط لأقرب مركز

3. تحديث المراكز

4. التكرار حتى الاستقرار (Convergence)

labels = kmeans.labels_

ما هو labels؟

• مصفوفة تحتوي رقم الكلاستر لكل نقطة.

• مثال:

• $[0, 0, 0, 1, 1, 1, 1, 0]$

• الرقم 0 أو 1 يدل على أي عنقود تنتمي إليه النقطة.

centroids = kmeans.cluster_centers_

ما هو cluster_centers_؟

• مصفوفة تحتوي الإحداثيات النهائية لمراكز العناقيد بعد انتهاء الخوارزمية.

• في مثالنا تقريبًا:

• $[[2.5, 1.75],$

• $[7.5, 5.0]]$

خلاصة تعليمية قصيرة

هذا الجزء ينشئ نموذج k-means ، يطبقه على البيانات، ثم يستخرج ناتج التقسيم (labels) ومراكز العناقيد النهائية (centroids).

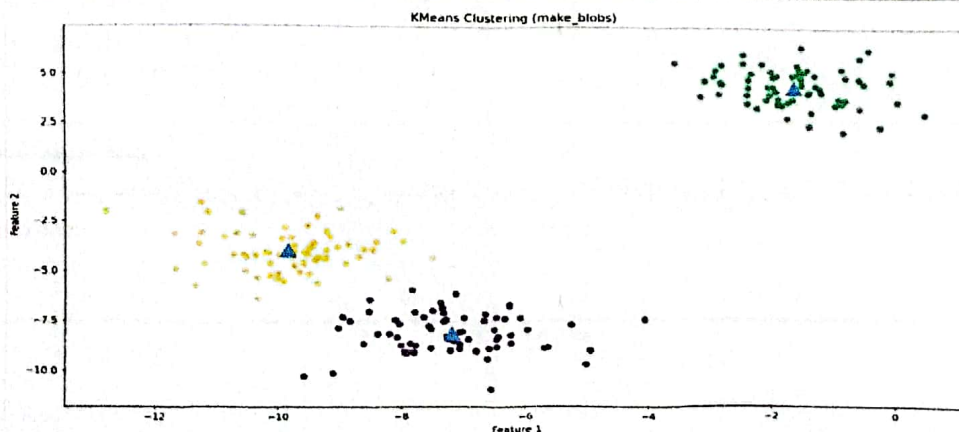
```

import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import make_blobs
from sklearn.cluster import KMeans

# -----
# Generate data
# -----
X, y = make_blobs(n_samples=200, centers=3, random_state=1)
# -----
# Apply KMeans
# -----
kmeans = KMeans(n_clusters=3, random_state=1)
kmeans.fit(X)
labels = kmeans.labels_
centroids = kmeans.cluster_centers_
# -----
# Printing
# -----
print("Cluster memberships:")
print(labels)
print("\nCluster centers:")
for i, c in enumerate(centroids):
    print(f"Cluster {i} center: {c}")
# -----
# Plotting
# -----
plt.figure()
plt.scatter(X[:, 0], X[:, 1], c=labels, marker='o')
plt.scatter(centroids[:, 0], centroids[:, 1],
            marker='^', s=250)
plt.title("KMeans Clustering (make_blobs)")
plt.xlabel("Feature 1")
plt.ylabel("Feature 2")
plt.grid(True)
plt.show()

```

الى هذه الخطوة يمكننا تسقي الكود هنا



◆ فكرة البرنامج العامة

البرنامج يقوم بـ:

1. توليد بيانات صناعية تحتوي على عنقايد باستخدام make_blobs
2. تطبيق خوارزمية k-Means لاكتشاف هذه العناقيد
3. طباعة نتائج التجميع
4. رسم النقاط ومراكز العناقيد

◆ شرح الأسطر بالتفصيل

1 استيراد المكتبات

```
from sklearn.datasets import make_blobs
from sklearn.cluster import KMeans
```

- make_blobs: تُستخدم لإنشاء بيانات اختبار تحتوي على عنقايد.
- KMeans: خوارزمية التجميع k-means من مكتبة scikit-learn.

2 توليد البيانات

- ```
X, y = make_blobs(random_state=1)
```
- X: مصفوفة البيانات (إحداثيات النقاط).
  - y: رقم العنقود الحقيقي لكل نقطة (Ground Truth).
  - random\_state=1: للتثبيت العشوائية والحصول على نفس البيانات في كل مرة.
  - y ⚠ لا يُستخدم في k-means لأنه خوارزمية غير مراقبة.

## 3 إنشاء نموذج k-Means

- ```
kmeans = KMeans(n_clusters=3)
```
- n_clusters=3: نطلب من الخوارزمية تقسيم البيانات إلى 3 عنقايد.

4 تدريب النموذج

- ```
kmeans.fit(X)
```
- يطبق خوارزمية k-means على البيانات X.
  - داخليًا يقوم بـ:
    - حساب المسافة
    - إسناد النقاط لأقرب مركز
    - تحديث المراكز
    - التكرار حتى الاستقرار

## 5 طباعة الانتماء إلى العناقيد

- ```
print("Cluster memberships:\n{}".format(kmeans.labels_))
```
- kmeans.labels_: مصفوفة تحتوي رقم العنقود لكل نقطة.
 - مثال:
 - [0 0 1 2 1 0 ...]

6 التنبؤ بالعنقود

- ```
print(kmeans.predict(X))
```
- تعيد نفس نتائج labels\_ تقريبًا.
  - تُستخدم عادةً لتحديد عنقود بيانات جديدة.